

VirtuEx 시스템 감사 보고서 (7차)

프로젝트: VirtuEx - 가상 자산 모의 거래 플랫폼 차수: 7차 감사 (종합 정기 감사) 작성일: 2026-03-03 작성: 시스템 감사팀

1. 개요

7차 감사는 VirtuEx 프로젝트 전체를 대상으로 한 종합 정기 감사입니다. 6차 감사(2026-03-02) 이후 수정된 사항을 검증하고, 백엔드 8개 마이크로서비스, 프론트엔드 Next.js 15 애플리케이션, 인프라/Docker 구성, 보안, 데이터 무결성, UX/접근성, 성능 등 전 영역을 코드 레벨에서 재검사했습니다.

1.1 감사 범위

영역	검사 항목	신규 이슈
6차 수정 검증	6차 75건 수정 현황	—
보안 — 인증/인가	JWT, 레이트 리밋, 내부 인증, CORS	13건
보안 — 입력 검증	DTO, 파라미터, WebSocket	5건
데이터 무결성	트랜잭션, 레이스 컨디션, 이벤트 소싱	11건
인프라/배포	Docker, CI/CD, 모니터링, 스크립트	9건
프론트엔드 — 코드 품질/버그	TypeScript, 로직 버그, 상태 관리	8건
프론트엔드 — 접근성/UX	ARIA, 키보드, 라우트 보호	8건
프론트엔드 — 성능/SEO	폴링, 이미지, 번들, 메타데이터	8건
합계		62건

2. 6차 감사 수정 현황

6차 감사에서 발견된 75건 중 수정 여부를 검증한 결과입니다.

2.1 수정 완료 (47건)

6차 #	항목	수정 내용
#1	JWT 풀백 인증 우회	UnauthorizedException 반환으로 변경
#3	SMS 비암호화적 난수	crypto.randomInt() 적용
#4	로그인 레이트 리밋	5req/60초 적용
#6	회원가입 레이트 리밋	3req/일 강화
#7	WS 익명 IP 제한	IP당 10연결, 연결당 5채널
#8	사용자 상태 캐시 TTL	60초 → 20초 단축
#12	주문 가격/수량 길이 제한	@MaxLength(50) 추가 (place-order)

#13	심볼 패턴 검증	@Matches(/^[A-Z0-9-]{1,20}\$/) 적용
#14	관리자 검색 파라미터	.slice(0, 100) 적용
#15	페이지네이션 최대값	Math.min(limit, 100) 적용
#16	입출금 최대 금액	@Max(1000000000) 적용
#25	Portfolio reserve/release 트랜잭션	\$transaction + FOR UPDATE 적용
#35	헬스체크 DB 프로빙	PrismaHealthIndicator 적용
#36	CI 보안 스캔 blocking	continue-on-error: false 적용
#75	HSTS 헤더	maxAge: 31536000, preload: true 적용

기타 32건: 프론트엔드/UX 마이너 이슈 수정 포함

2.2 부분 수정 (6건)

6차 #	항목	현황
#2	주민번호 솔트 하드코딩	VO 수정 완료, 하지만 auth.service.ts에서 고정 솔트 전달 (7차 #1)
#5	주문 API 레이트 리밋	60req/min 적용 (여전히 높음)
#20	주문 취소 자금 해제 레이스	롤백 로직 추가, 단일 트랜잭션 미적용
#30	서비스 URL 하드코딩	order-engine getOrThrow() 적용, proxy.service 미적용 (7차 #32)
#34	요청 ID 추적	미들웨어 추가, 다운스트림 전파 미구현 (7차 #34)
#37	Docker 이미지 버전	Kafka, Grafana 등 고정, PostgreSQL/Redis 미고정 (7차 #30)

2.3 미수정 (22건)

6차 #9, #10, #11, #17, ~~#19, #21, #24, #26, #29, #31~~ #33, #38, #69~#71, #73 — 장기 과제 포함

3. 보안 감사 — 인증/인가 (13건)

#	위험 도	항목	위치
1	● 상	주민번호 암호화 솔트 실질적 하드코딩	auth.service.ts:99-100
2	● 중	verify-sms 레이트 리밋 미흡 (10req/min)	auth-proxy.controller.ts:65
3	● 중	check-duplicate 레이트 리밋 부재	auth-proxy.controller.ts:216
4	● 중	비밀번호 재설정 복잡도 검증 없음	forgot-password.dto.ts:22

5	● 중	TOTP 암호화에 JWT_SECRET 재사용	totp.service.ts:35
6	● 중	Email 컨트롤러 타이밍 공격 취약	email.controller.ts:47
7	● 중	INTERNAL_SERVICE_SECRET 빈 문자열 기 본값	proxy.service.ts:22
8	● 중	내부 서비스 CORS 외부 허용	portfolio/main.ts, order- engine/main.ts
9	● 중	Admin Gateway 역할 검증 미적용	admin-proxy.controller.ts:29
10	● 중	채팅 통계 프록시 x-internal-token 누락	statistics-proxy.controller.ts:229
11	● 중	마이페이지에서 Admin API 직접 호출	프론트엔드 mypage/page.tsx:65
12	● 하	리더보드 응답에 userId 노출	portfolio-proxy.controller.ts:114
13	● 하	Swagger 문서 프로덕션 환경 노출	api-gateway/main.ts:74

상세 설명:

- **#1:** resident-number.vo.ts 는 레코드별 랜덤 솔트로 수정되었으나, auth.service.ts:99 에서 configService.get('ENCRYPTION_SALT', 'virtuex-salt') 로 고정 솔트를 전달하여 랜덤 솔트 기능이 무효화됨. getOrThrow() 로 변경하거나 솔트 파라미터 제거 필요
- **#2:** /login/verify-sms 가 10req/60초 허용. SMS 코드 6자리(100만 경우) 대비 분당 10회 시도는 브루트포스에 취약. 서비스 내부 5회 제한 존재하나 레이트 리밋도 5req/60초로 강화 필요
- **#3:** /check-duplicate 엔드포인트에 별도 레이트 리밋 없음 (글로벌 100req/min만). 사용자 열거(User Enumeration) 공격에 활용 가능
- **#4:** ResetPasswordDto.newPassword 에 @MinLength(8) 만 적용. register.dto.ts 는 소문자/숫자/특수문자 @Matches 3개 검증 — 비밀번호 재설정에서 더 강한 비밀번호 설정 가능
- **#5:** totp.service.ts 에서 TOTP 시크릿 암호화 키를 JWT_SECRET 에서 파생. JWT_SECRET 유출 시 저장된 TOTP 시크릿도 복호화 가능. 별도 TOTP_ENCRYPTION_KEY 환경변수 분리 필요
- **#6:** notification 서비스의 email.controller.ts 만 token !== secret 문자열 직접 비교 사용. 타 서비스의 InternalAuthGuard 는 모두 timingSafeEqual 적용
- **#7:** proxy.service.ts , jwt.strategy.ts 에서 INTERNAL_SERVICE_SECRET 기본값이 빈 문자열. 환경변수 누락 시 빈 토큰으로 요청 발송. getOrThrow() 필요
- **#8:** portfolio, order-engine 등 내부 전용 서비스가 enableCors() 로 외부 브라우저 접근 허용. API Gateway 만 통해야 하므로 CORS 비활성화 권장
- **#9:** AdminProxyController 에 JwtAuthGuard 만 적용, 역할 검증 없이 user-auth로 프록시. 비관리자 트래픽이 user-auth까지 도달
- **#10:** 채팅 통계 프록시 요청에 x-internal-token 헤더 미포함. chat 서비스의 InternalAuthGuard 에 의해 차단될 수 있음
- **#11:** 프론트엔드 마이페이지의 알림 토글이 /api/admin/settings 엔드포인트를 직접 호출. 사용자 전용 엔드포인트 사용 필요

- #12: 리더보드 API 응답에 다른 사용자의 내부 UUID 노출. 마스킹 권장
- #13: `SwaggerModule.setup()` 이 `NODE_ENV` 조건 없이 항상 활성화

4. 보안 감사 — 입력 검증 (5건)

#	위험도	항목	위치
14	상	주문 수정 DTO MaxLength 누락	<code>modify-order.dto.ts:11-18</code>
15	중	<code>tradingStats</code> days 파라미터 상한 없음	<code>order.controller.ts:154</code>
16	중	회원가입 주소/우편번호 필드 검증 미흡	<code>register.dto.ts:49-57</code>
17	중	채팅 방 이름 변경 body 검증 없음	<code>chat.controller.ts:143</code>
18	하	x-user-id 헤더 UUID 형식 미검증	<code>order.controller.ts:165</code>

상세 설명:

- #14: `place-order.dto.ts` 에는 6차 수정으로 `@MaxLength(50)` 추가되었으나, `modify-order.dto.ts` 의 `price` , `quantity` 필드는 누락. 수만 자리 문자열로 파싱 DoS 가능
- #15: `/stats/trading?days=99999` 요청 시 DB에 대량 쿼리 발생. `Math.min(daysNum, 365)` 상한 필요
- #16: `residentNumber` 에 `@Length(13,13)` 또는 패턴 없음, `address / addressDetail` 에 `@MaxLength` 없음, `zipCode` 에 `@Matches(/^d{5}$/)` 없음
- #17: `renameRoom()` body에 DTO 미적용 — 방 이름 길이/형식 검증 없이 직접 `body.name` 사용
- #18: 내부 서비스의 `validateUserId()` 가 존재 여부만 확인하고 UUID 형식 미검증. Prisma parameterized query로 SQL 인젝션은 방지되나 명시적 검증 권장

5. 데이터 무결성 감사 (11건)

#	위험도	항목	위치
19	상	<code>deposit/withdraw</code> FOR UPDATE 락 누락	<code>balance.service.ts:158-232</code>
20	상	<code>settleBuy/Sell Holding</code> FOR UPDATE 누락	<code>balance.service.ts:407-545</code>
21	상	SELL 주문 취소 시 Holdings 복원 미구현	<code>order.service.ts:300-322</code>
22	상	주문 수정(<code>modifyOrder</code>) 자금 미조정	<code>order.service.ts:442-505</code>
23	상	조건부 주문(<code>checkTriggers</code>) 이중 발동	<code>order.service.ts:512-624</code>
24	상	Outbox 발행자(Publisher) 미구현	<code>event-store.service.ts:184-203</code>
25	중	정산 실패 DLQ/재처리 메커니즘 부재	<code>order.service.ts:870-922</code>
26	중	이벤트 버전 계산 로직 오류 가능성	<code>order.service.ts:183</code>
27	중	KRW 심볼(.KS) <code>reserve-holdings</code> 정규식 불일치	<code>internal.controller.ts:23</code>
28	중	Account 잔고 DB 레벨 음수 방지 제약 없음	<code>portfolio/schema.prisma:14</code>
29	하	Transaction <code>referenceId</code> unique 제약 없음	<code>portfolio/schema.prisma:59</code>

상세 설명:

- **#19:** `deposit()` 과 `withdraw()` 에서 `ensureAccount()` 후 잔고를 JS 변수에 캡처하고, `$transaction` 내에서 갱신. 읽기와 갱신 사이에 다른 요청이 끼어들면 Lost Update 발생. `reserveFunds()` / `settleBuy()` 등은 FOR UPDATE 적용했으나 `deposit()` / `withdraw()` 는 미적용
- **#20:** `settleBuy()` / `settleSell()` 내에서 Account는 FOR UPDATE 락을 거나, Holding 레코드는 일반 `findUnique()` 로 읽음. 동일 종목 동시 체결 시 하나의 변경이 다른 것을 덮어쓸 수 있음
- **#21:** 주문 취소 시 BUY 주문은 예약 자금 해제하지만, SELL 주문은 Holdings 복원 로직 없음. SELL 취소 시 보유량이 영구 차감 상태로 유지
- **#22:** `modifyOrder()` 에서 가격/수량 변경 시 자금 재예약(BUY) 또는 Holdings 재검증(SELL) 미수행. 가격 인상 시 추가 자금 예약 없이 더 큰 총 비용 발생, 가격 인하 시 과잉 예약 자금 미해제
- **#23:** `checkTriggers()` 에서 `triggered: false` 로 조회 후 `triggered: true` 업데이트 사이에 또 다른 호출이 같은 주문을 읽어 이중 체결 가능. `UPDATE ... WHERE triggered = false RETURNING *` 원자적 업데이트 필요
- **#24:** `event_outbox` 테이블과 `getUnpublishedOutboxEntries()` , `markOutboxPublished()` 메서드가 구현되어 있으나, 실제로 미발행 Outbox를 Kafka에 발행하는 스케줄러/서비스가 없음. Outbox 패턴이 절반만 구현됨
- **#25:** `settleTrade()` 에서 포트폴리오 HTTP 요청 실패 시 로그만 남기고 계속 진행. 이벤트 스토어에는 체결 기록, 포트폴리오는 미갱신 — 불일치 상태 발생. DLQ 또는 재처리 테이블 필요
- **#26:** `expectedVersion` 계산이 `order.version - uncommittedEvents.length + 1 + indexOf(event)` 패턴으로 복잡. 복수 이벤트 루프 + `clearUncommittedEvents()` 호출 시 버전 꼬임 가능
- **#27:** `SYMBOL_REGEX = /^[A-Z]{2,10}(-USD)?$/` 로 `005930.KS` 같은 한국 주식 심볼 매칭 불가. `balance.service.ts` 의 `isUsdSymbol()` 은 `.KS` 인식 — 불일치
- **#28:** `availableCash` , `reservedCash` 에 DB 레벨 CHECK (`available_cash >= 0`) 제약 없음. 앱 로직 버그 시 음수 잔고 입력 가능
- **#29:** `Transaction.referenceId` 에 unique 제약 없음. `settleBuy()` / `settleSell()` 재시도 시 동일 `tradeId`로 중복 정산 트랜잭션 생성 가능

6. 인프라/배포 감사 (9건)

#	위험도	항목	위치
30	● 중	PostgreSQL/Redis 이미지 마이너 버전 미고정	<code>docker-compose.yml</code> L7, L38
31	● 중	Redis Commander 인증 없이 Redis 접근	<code>docker-compose.yml</code> L167
32	● 중	<code>proxy.service</code> 서비스 URL localhost 기본값	<code>proxy.service.ts:24-30</code>
33	● 중	요청 ID(x-request-id) 다운스트림 미전파	<code>proxy.service.ts forward()</code>
34	● 중	전역 예외 필터 마이크로서비스 미적용	각 서비스 <code>main.ts</code>
35	● 중	테스트 커버리지 임계값 미설정	<code>ci.yml</code> , Jest 설정
36	● 중	컨테이너 이미지 취약점 스캔 없음	<code>.github/workflows/ci.yml</code>
37	● 하	Prometheus scrape 타겟 <code>host.docker.internal</code> 고정	<code>prometheus.yml</code> L13-49
38	● 하	<code>start-all.sh</code> readiness probe 미적용	<code>scripts/start-all.sh:382</code>

상세 설명:

- **#30:** postgres:16-alpine , redis:7-alpine 은 마이너/패치 미지정. 자동 업데이트 시 호환성 문제 가능. postgres:16.8-alpine , redis:7.4-alpine 등 정확한 버전 지정 권장
- **#31:** redis-commander 가 패스워드 없이 Redis 접근. Redis 자체는 `--requirepass` 설정 있으나 Commander 연결에 미포함
- **#32:** proxy.service.ts 에서 `configService.get('USER_AUTH_PORT', 3007)` 패턴 사용. 환경변수 누락 시 localhost:3007 로 연결. `getOrThrow()` 필요 (6차 #30 잔존)
- **#33:** request-id.middleware.ts 에서 x-request-id 생성하지만, proxy.service.ts 의 `forward()` 가 다운스트림 서비스에 미전파. 분산 추적이 Gateway 경계에서 끊김
- **#34:** AllExceptionsFilter 가 api-gateway에만 등록. 내부 서비스 간 통신에서 NestJS 기본 에러 포맷 노출 (스택 트레이스 포함 가능)
- **#35:** CI에 test:cov 스텝 있으나 `coverageThreshold` 미설정. 테스트 파일이 서비스당 ~2개 수준. 70%+ 커버리지 목표 설정 권장
- **#36:** npm audit --prod 는 npm 의존성 취약점만 검사. Docker 이미지 OS 레벨 취약점(CVE) 미감지. Trivy 또는 Snyk container scan 추가 권장
- **#37:** prometheus.yml 의 모든 scrape 타겟이 host.docker.internal 로 하드코딩. macOS/Windows 에서만 동작, Linux 프로덕션 서버에서 미동작
- **#38:** start-all.sh 에서 포트 LISTEN 상태만 확인. order-engine `onModuleInit()` 완료 전 주문 유입 가능. /health/ready HTTP 응답으로 readiness 확인 필요

7. 프론트엔드 감사 — 코드 품질/버그 (8건)

#	위험도	항목	위치
39	상	useRecentTrades side 판별 항상 'BUY' 반환	useMarket.ts:194
40	상	Swagger URL localhost 하드코딩	page.tsx:44
41	중	useCandlesticks 중복 aggregateCandles 호출	useMarket.ts:135-138
42	중	leaderboard 랭크 변동 시뮬레이션 데이터	leaderboard/page.tsx:82-91
43	중	Header 로그아웃 모달 ConfirmModal 미활용	Header.tsx:491-517
44	중	auth.ts accessToken 미퍼시스트 race condition	stores/auth.ts:37-40
45	하	not-found.tsx / error.tsx에서 Next.js Link 미사용	not-found.tsx:31, error.tsx:138
46	하	max-w-full + max-w-[calc] 중복 클래스	Header.tsx:494, mypage/page.tsx:502

상세 설명:

- **#39:** `side: (t.buyerId === t.sellerId ? 'BUY' : 'BUY')` — 삼항연산자 양 분기가 모두 'BUY' . 체결 내역에서 SELL 거래가 전부 BUY로 표시됨. 올바른 로직으로 수정 필요
- **#40:** 랜딩 페이지 Swagger 버튼이 `window.open('http://localhost:3000/api-docs')` . 프로덕션에서 항상 localhost를 열어 접속 불가. NEXT_PUBLIC_API_URL 활용 필요
- **#41:** useCandlesticks 내 조건 분기 양측이 동일한 aggregateCandles() 호출. 주석("Skip aggregation if data already in correct interval")과 코드 불일치
- **#42:** 리더보드 랭크 변동이 실제 데이터가 아닌 userId 기반 시드 난수로 생성. 사용자에게 허위 정보 표시

- **#43:** 앱에 `ConfirmModal` 공통 컴포넌트가 있으나 `Header` 로그아웃은 인라인으로 중복 구현.
`role="dialog"` , `focus trap` , `aria-labelledby` 미적용
- **#44:** `accessToken` 이 `persist`에서 의도적 제외. 새로고침 시 `isAuthenticated: true` 이나 `accessToken: null` — `refresh` 완료 전 병렬 요청에서 `Authorization` 헤더 누락 가능
- **#45:** `<a href="/">` 직접 사용으로 전체 페이지 리로드 발생. `<Link href="/">` 로 교체 필요
- **#46:** `max-w-full` 과 `max-w-[calc(100vw-2rem)]` 중복 적용. `Tailwind`에서 후자만 유효하므로 전자 제거 필요

8. 프론트엔드 감사 — 접근성/UX (8건)

#	위험도	항목	위치
47	상	Next.js middleware 부재 (서버사이드 보호 없음)	frontend/src/ 전체
48	상	관리자 페이지 클라이언트 사이드 <code>role</code> 체크만 존재	admin/users/page.tsx:194
49	중	다수 모달 <code>aria-labelledby-describedby</code> 미적용	mypage/page.tsx, Header.tsx
50	중	Dashboard 탭 ARIA <code>role</code> 누락	dashboard/page.tsx:181-196
51	중	admin/users 테이블 행 키보드 접근 불가	admin/users/page.tsx:333
52	중	OrderSheet 닫기 버튼 <code>aria-label</code> 누락	OrderSheet.tsx:72
53	하	<code>lang="ko"</code> 하드코딩 (다국어 앱과 불일치)	layout.tsx:34
54	하	BottomNav 터치 타겟 높이 미흡 (48px)	BottomNav.tsx:24

상세 설명:

- **#47:** `middleware.ts` 파일 없음. 모든 보호 페이지가 클라이언트 `AuthGuard` 와 `useEffect` 리다이렉트에만 의존. SSR 시 보호된 페이지 HTML이 잠깐 노출 가능. 검색 엔진 크롤러에 민감한 UI 구조 노출
- **#48:** 관리자 페이지가 `user.role !== 'ADMIN'` 클라이언트 체크로만 보호. JS 비활성화 또는 로컬스토리지 조작으로 UI 접근 가능 (API 자체 권한 체크는 존재)
- **#49:** `mypage` 비밀번호 변경 모달, 2차 확인 모달에 `role="dialog"` `aria-modal="true"` 있으나 `aria-labelledby` 없음. `Header` 로그아웃 모달은 `role="dialog"` 자체 미적용
- **#50:** 대시보드 탭 UI에 `role="tab"` , `aria-selected` , `role="tablist"` wrapper 없음. 스크린리더에서 일반 버튼으로 인식
- **#51:** `<tr onClick>` 방식으로 클릭 처리. `tabIndex={0}` , `onKeyDown` (Enter/Space) 미처리. 키보드 사용자 행 클릭 불가
- **#52:** 아이콘만 있는 닫기 버튼에 `aria-label` 없음. `BottomSheet.tsx` 는 올바르게 적용되어 있어 불일관
- **#53:** `<html lang="ko">` 가 고정. 한국어/영어 전환 앱이지만 `lang` 속성 미동기화. 스크린리더/SEO 영향
- **#54:** `BottomNav` 전체 높이 48px, 각 아이템 `py-1` 포함. Apple HIG/WCAG 권고 최소 44×44px 터치 영역에 근접하여 소형 기기에서 미스터리 가능

9. 프론트엔드 감사 — 성능/SEO (8건)

#	위험도	항목	위치
55	중	REST polling + WebSocket 동시 실행	useMarket.ts hooks 전반
56	중	<code>useCandlesticks</code> <code>fetchLimit</code> 50,000개	useMarket.ts:90-92

57	● 중	무거운 컴포넌트 dynamic import 미적용	dashboard/page.tsx 등
58	● 중	CSP에 unsafe-eval 허용	next.config.ts:30
59	● 중	주요 페이지 metadata 미설정	news, leaderboard, community 등
60	● 하	portfolio 1초마다 setNow 리렌더	portfolio/page.tsx:48-52
61	● 하	admin/health localhost 포트 하드코딩 노출	admin/health/page.tsx:623
62	● 하	next/image unoptimized 플래그 사용	page.tsx:128

상세 설명:

- **#55:** 자산 상세 페이지에서 useAssetPrice (3초), useOrderBook (3초), useCandlesticks (5초), useRecentTrades (5초) REST polling과 WebSocket이 동시 실행. WebSocket 연결 시 REST polling 비활성화 또는 간격 대폭 확대 필요
- **#56:** 1일 캔들 요청 시 $limit * 1440 = 50,000$ 개의 1분 캔들을 서버에서 fetch 후 클라이언트에서 집계. 서버 사이드 집계 또는 별도 캔들 엔드포인트 필요
- **#57:** CandlestickChart (lightweight-charts 라이브러리), ChatPanel, NotificationBell 등 무거운 컴포넌트에 dynamic import 미적용. AiInsights 만 dynamic() 적용 상태
- **#58:** script-src 'self' 'unsafe-eval' 'unsafe-inline' — unsafe-eval 은 XSS 공격 표면 확대. 개발 환경 Next.js fast refresh용이나 프로덕션에서도 동일 CSP 적용. 환경별 분기 필요
- **#59:** /news, /leaderboard, /announcements, /community, /help, /mypage, /asset/[symbol], /admin/* 페이지에 개별 metadata 없음. Next.js 15 generateMetadata() 활용 권장
- **#60:** setInterval(() => setNow(Date.now()), 1000) — 1초마다 컴포넌트 리렌더. lastUpdatedText 가 분 단위이므로 60초 interval 충분
- **#61:** admin health 페이지에 http://localhost:\${port} 내부 엔드포인트 하드코딩 노출. 환경변수 기반 동적 표시 필요
- **#62:** next/image 에 unoptimized 플래그로 실질적 이미지 최적화 비활성화. 마귀 내 모든 아이콘에 priority 적용

10. 종합 평가

10.1 위험도별 분류

위험도	건수	비율
● 상 (Critical)	10건	16.1%
● 중 (Medium)	35건	56.5%
● 하 (Low/Info)	17건	27.4%
합계	62건	100%

10.2 영역별 성숙도 (6차 대비)

항목	6차	7차	변화	비고
인증/인가	3.5/5	4.0/5	▲+0.5	JWT 폴백, 레이트 리밋, WS 제한 수정 완료

입력 검증	3.5/5	4.0/5	▲+0.5	주문 DTO, 관리자 파라미터 검증 강화
데이터 무결성	3.0/5	3.0/5	—	reserve/release 개선, deposit/withdraw 미흡
인프라/배포	4.0/5	4.0/5	—	헬스체크/HSTS 수정, 메트릭 미구현 지속
프론트엔드 접근성/UX	3.0/5	3.0/5	—	기존 이슈 일부 수정, 신규 이슈 다수 발견
프론트엔드 코드 품질	3.5/5	3.5/5	—	버그 2건 발견, 전반적 코드 품질 유지
SEO/성능	2.5/5	2.5/5	—	metadata/sitemap 미진전
성능/확장성	3.5/5	3.5/5	—	폴링/번들 최적화 미흡
종합	3.3/5	3.4/5	▲+0.1	

10.3 누적 감사 현황

차수	유형	발견	수정	스킵	미수정
1차	신규 감사	36	36	0	0
2차	신규 감사	50	45	4	1
3차	수정 검증	0	—	—	—
4차	신규 감사	79	79	0	0
5차	수정 검증	0	—	—	—
6차	종합 심층 감사	75	47	0	28
7차	종합 정기 감사	62	미수정	0	62
누적		302	207	4	91

10.4 즉시 수정 권장 사항 (상 위험도 10건)

1. 주민번호 솔트 실질적 하드코딩 (#1) — `auth.service.ts` 고정 솔트 전달 제거
2. 주문 수정 DTO MaxLength (#14) — `modify-order.dto.ts` 에 `@MaxLength(50)` 추가
3. deposit/withdraw FOR UPDATE (#19) — 트랜잭션 내 `SELECT ... FOR UPDATE` 적용
4. Holding FOR UPDATE (#20) — `settleBuy/Sell` 내 Holding 읽기에 row lock 적용
5. SELL 취소 Holdings 복원 (#21) — SELL 주문 취소 시 보유량 복원 로직 추가
6. 주문 수정 자금 미조정 (#22) — `modifyOrder`에 자금 재예약/해제 로직 추가
7. checkTriggers 이중 발동 (#23) — 원자적 `UPDATE ... RETURNING` 패턴 적용
8. Outbox 발행자 미구현 (#24) — `OutboxRelayService` 스케줄러 구현
9. useRecentTrades side 버그 (#39) — 삼항연산자 수정
10. Next.js middleware 부재 (#47) — 서버사이드 라우트 보호 미들웨어 추가

10.5 6차 대비 주요 개선 사항

- 인증 보안 대폭 강화: JWT 토큰 차단, 로그인/회원가입 레이트 리밋, WS IP 제한, 캐시 TTL 단축 등 6차 상 위험도 보안 이슈 전량 수정
- 입력 검증 체계화: 주문 DTO 길이 제한, 관리자 검색 파라미터 검증, 입출금 금액 상한, 심볼 패턴 검증 추가
- 트랜잭션 안전성 향상: Portfolio reserve/release에 `$transaction + FOR UPDATE` 패턴 적용
- 인프라 보안 강화: HSTS 헤더, 헬스체크 DB 프로빙, CI 보안 스캔 blocking 적용

10.6 프로젝트 강점

- **마이크로서비스 아키텍처:** 8개 서비스 분리 + CQRS/이벤트 소싱 패턴
- **인증 체계:** JWT + 2FA(TOTP) + 계정 잠금 + timing-safe 비교 + 강화된 레이트 리밋
- **인프라 구성:** Docker Compose + Prometheus + Grafana + Jaeger + 구조화된 헬스체크
- **프론트엔드 설계:** TypeScript strict 모드, Zustand + React Query, WebSocket 실시간 가격
- **코드 품질:** ESLint, Prettier, Husky, `console.log` 없음, `as any` / `@ts-ignore` 없음
- **접근성:** ConfirmModal/BottomSheet focus trap, skip-to-content, skeleton loading 일관 구현
- **i18n 지원:** 한국어/영어 다국어 구현, SSR hydration flicker 방지

10.7 결론

6차 감사 이후 **47건이 수정 완료**되어 특히 인증/인가 및 입력 검증 영역에서 큰 개선이 이루어졌습니다. 6차의 상 위험도 8건이 모두 해결된 점은 프로젝트의 보안 대응 능력을 보여줍니다.

7차 감사에서는 62건의 신규 이슈를 발견했으며, 특히 **데이터 무결성 영역**(deposit/withdraw 트랜잭션, Holdings 동시성, 주문 수정 자금 조정, Outbox 미구현)이 가장 시급한 개선 대상입니다. 프론트엔드에서는 `useRecentTrades` side 버그, Next.js middleware 부재, 과도한 REST polling이 핵심 우선순위입니다.

데이터 무결성 상 위험도 6건은 금융 트랜잭션의 정확성에 직접 영향하므로 **즉시 수정**을 권장합니다.

본 보고서는 VirtuEx 시스템의 7차 종합 정기 감사 결과입니다. 2026-03-03 기준 소스 코드를 대상으로 합니다.